

Práctica 1.1 - Desarrollo de software para tienda de videojuegos

Planificación del proyecto de desarrollo de software

1. Análisis de Requerimientos y Diseño

1.1 Requisitos Funcionales:

- **Gestión de ventas y facturación:** registrar cada venta y emitir la factura correspondiente.
- **Registro de venta por empleado:** Asociar cada venta al trabajador que la realiza, para un control individualizado.
- **Gestión de inventario:** consultar y actualizar el stock disponible en almacén en tiempo real.
- **Compatibilidad con hardware:** integración con lector de códigos de barras y con lector/datáfono.
- **Gestión de precios:** alta, baja y modificación de precios de los productos.
- **Gestión de datos:** almacenar la información de trabajadores (DNI, nombre, apellidos, nº de la Seguridad Social, fecha de nacimiento, teléfono, localidad) y de productos (código, marca, nombre comercial, precio, cantidad).

1.2 Requisitos No Funcionales:

- **Rendimiento (eficiencia):** tiempos de respuesta óptimos en consultas de stock, precio y emisión de factura.
- **Concurrencia controlada (fiabilidad):** el sistema no procesará dos peticiones de forma simultánea, aunque existan varios equipos operando a la vez (acceso exclusivo/serializado a los recursos compartidos, p. ej. mediante bloqueos sobre la base de datos).
- **Software libre:** todas las herramientas, lenguajes y bibliotecas empleadas deben ser de código abierto.

1.3 Diseño inicial de la aplicación

Se propone una arquitectura modular en capas (presentación, lógica de negocio y datos), organizada en los siguientes módulos, que podrán desarrollarse de forma independiente:

- **Módulo de Ventas y Facturación:** registra ventas y genera facturas
- **Módulo de Inventario:** gestiona el stock y las altas/bajas de productos.
- **Módulo de Empleados:** mantiene los datos de los trabajadores y sus ventas.
- **Módulo de Precios:** centraliza la actualización de tarifas.

- **Módulo de Periféricos:** capa de integración con el lector de códigos de barras y el datáfono.

Como almacén de datos se plantea una base de datos relacional (p. ej. PostgreSQL) con, al menos, las entidades Empleado, Producto y Venta, relacionadas mediante claves foráneas que permitan asociar cada venta a un empleado y a los productos vendidos.

1.4 Modelo de ciclo de vida elegido

Se propone el **modelo en cascada con retroalimentación**. Se justifica porque el cliente ha definido un conjunto de requisitos claro, cerrado y bien delimitado (facturación, inventario, empleados, hardware y precios), lo que encaja con un modelo secuencial; a la vez, la retroalimentación entre fases permite corregir errores de análisis o diseño detectados durante la codificación o las pruebas sin tener que reiniciar todo el proyecto, algo importante dado que la aplicación debe integrarse con hardware específico (lector de códigos y datáfono), donde suelen surgir ajustes durante la implementación. Un modelo puramente iterativo-incremental se descarta por ser un proyecto de alcance acotado y no un producto en evolución continua, y la espiral se considera excesiva para el tamaño y riesgo del proyecto.

2. Planificación de la Codificación

2.1 Lenguaje de programación

Se elige **Java** (distribución libre OpenJDK), lenguaje orientado a objetos, compilado a ByteCode e interpretado por la JVM, lo que aporta portabilidad entre los distintos equipos de la tienda sin depender de una arquitectura concreta. Su modelo de clases y objetos facilita representar de forma natural las entidades del negocio (Empleado, Producto, Venta, Factura) y su carácter de software libre cumple con la exigencia del cliente.

2.2 Herramientas de desarrollo

Herramienta	Elección	Justificación
Entorno de desarrollo (IDE)	IntelliJ community edition	Gratuitos, con autocompletado, depurador integrado y soporte nativo para Java.
Compilador (código fuente -> objeto)	javac (OpenJDK)	Traduce el código fuente (.java) a código objeto o bytecode (.class), multiplataforma.
Intérprete / máquina virtual (objeto -> ejecución)	JVM (OpenJDK)	Interpreta el bytecode y genera el código ejecutable en cada equipo de la tienda

Herramienta	Elección	Justificación
Framework de desarrollo	Spring	Acelera el desarrollo de la lógica de negocio y la capa de acceso a datos; muy usado en gestión empresarial.
Base de datos	PostgreSQL	Motor relacional libre, robusto para el volumen de datos de una tienda.
Gestión de dependencias	Maven	Automatiza la compilación, empaquetado y gestión de librerías del proyecto.
Control de versiones	Git	Permite el trabajo colaborativo y el control de cambios sobre el código fuente.
Pruebas unitarias	JUnit	Estándar libre para pruebas unitarias en proyectos Java.

Para la comunicación con el lector de códigos de barras y el datáfono se plantea el uso de bibliotecas Java compatibles con dispositivos de punto de venta (p. ej. jPOS), evitando así software propietario y facilitando el cumplimiento del requisito de software libre.

3. Planificación de las Fases del Ciclo de Vida

Fase	Objetivos	Estrategias
Análisis	Obtener una especificación de requisitos (ERS) completa, sin ambigüedades y validada por el cliente.	Reuniones con SoftWareHouse; separación de requisitos funcionales y no funcionales; estimación inicial de tiempos (diagrama de Gantt).
Diseño	Definir la arquitectura, los módulos, el modelo de datos y la interfaz de usuario.	Diagramas de clases y de la base de datos; diseño de pantallas del TPV; elaboración del plan de pruebas.
Codificación	Implementar cada módulo siguiendo el diseño acordado.	Desarrollo por módulos en Java/Spring Boot; revisiones de código; control de versiones con Git; pruebas unitarias con JUnit sobre cada clase.
Pruebas	Verificar que el sistema integrado cumple la ERS y funciona sin errores.	Pruebas de integración entre los módulos de ventas, inventario y hardware; comparación de

		resultados obtenidos frente a los esperados.
Implantación	Poner en marcha la aplicación en la tienda sin interrumpir la actividad comercial.	Instalación en los equipos, conexión con el lector de códigos y el datáfono, carga inicial del inventario y formación breve al personal.
Mantenimiento	Garantizar la fiabilidad de la aplicación a largo plazo.	Mantenimiento correctivo ante errores, evolutivo ante nuevas peticiones del cliente y adaptativo ante cambios de hardware o normativa.

En conjunto, la planificación combina el orden y la trazabilidad del modelo en cascada con retroalimentación entre fases, de modo que un error detectado en pruebas o implantación pueda corregirse volviendo al diseño o al análisis sin reiniciar el proyecto completo.